Phase 1 — Foundations of Cryptography

The Complete Foundation for Building ECDAT

Version: 1.0

Project: Enterprise Cryptographic Discovery & Analysis Tool (ECDAT)

Organization: National Technical Research Organisation (NTRO)

Smart India Hackathon 2026

Table of Contents

---

1. Introduction

Cryptography is the science of securing information from unauthorized access while ensuring that only intended parties can understand or modify it.

For centuries, cryptography was primarily used for military communication. Today, it forms the backbone of the Internet, banking systems, cloud computing, healthcare, digital identity, national defense, and enterprise infrastructure.

Every HTTPS website, online payment, software update, cloud login, VPN connection, and encrypted database relies on cryptography.

The NTRO ECDAT problem is **not about inventing new encryption algorithms**. It is about **discovering where cryptography is already used**, understanding whether it is secure against future quantum threats, and helping organizations migrate safely.

To build ECDAT, the team must first understand modern cryptography from the ground up.

---

2. What is Cryptography?

Cryptography is the discipline of protecting information using mathematical algorithms.

It provides mechanisms to:

- Keep data confidential.
- Verify identities.
- Ensure data has not been altered.
- Prevent impersonation.
- Secure communication over untrusted networks.

At its core, cryptography transforms readable information (**plaintext**) into unreadable information (**ciphertext**) using a cryptographic algorithm and one or more keys.

```
Plaintext
     |
     ▼
Encryption Algorithm
     |
     ▼
Ciphertext
     |
     ▼
Decryption Algorithm
```

```
         |
         ▼
    Original Plaintext
```

Without the correct key, recovering the original information should be computationally infeasible.

---

3. Why Cryptography Exists

Imagine sending your ATM PIN through the internet without encryption.

Anyone monitoring network traffic could immediately read it.

Similarly:

- Hospitals transmit patient records.
- Governments exchange classified information.
- Banks process billions of transactions.
- Enterprises store trade secrets.
- Cloud providers host customer workloads.

All of this data travels across networks that are not inherently trusted.

Cryptography allows these communications to remain secure even if attackers can observe the network.

---

4. Evolution of Cryptography

Ancient Cryptography

Early civilizations relied on substitution and transposition ciphers.

Examples include:

- Caesar Cipher
- Spartan Scytale
- Atbash Cipher
- Vigenère Cipher

These systems depended on secrecy of the method rather than mathematical complexity.

Mechanical Cryptography

The twentieth century introduced mechanical encryption machines such as:

- Enigma Machine
- Lorenz Cipher

These dramatically increased encryption complexity but were eventually broken through cryptanalysis and computation.

Modern Cryptography

Modern cryptography is based on computational hardness assumptions.

Instead of hiding the algorithm, security relies on mathematical problems believed to be computationally difficult.

Examples include:

- Integer factorization
- Discrete logarithm
- Elliptic Curve Discrete Logarithm Problem
- Lattice problems (used in Post-Quantum Cryptography)

This approach follows **Kerckhoffs's Principle**:

> A cryptographic system should remain secure even if everything about the system except the key is public knowledge.

---

5. Security Goals

Modern cryptography supports several fundamental security properties.

Confidentiality

Only authorized users should access the information.

Example:

Customer passwords stored in encrypted databases.

Integrity

Data must not be modified without detection.

Example:

Software downloads verified using cryptographic hashes.

Availability

Authorized users should have access when needed.

While availability is primarily a system property, cryptographic infrastructure such as PKI and key servers must remain available.

Authentication

Verifies the identity of users, systems, or devices.

Example:

Logging into an enterprise portal using certificates.

Authorization

Determines what authenticated users are allowed to do.

Non-Repudiation

Ensures that the sender cannot deny having performed an action.

Example:

Digitally signed contracts.

---

6. Types of Cryptography

Modern cryptography is divided into three major categories.

```
Cryptography
|
├── Symmetric Encryption
|
├── Asymmetric Encryption
|
└── Hash Functions
```

Each serves different purposes.

---

7. Symmetric Cryptography

Symmetric encryption uses a **single shared secret key** for both encryption and decryption.

```
Shared Secret Key
       |
       ▼
```

```
    Encrypt
           |
    Ciphertext
           |
    Decrypt
           ▼
    Plaintext
```

Advantages

- Extremely fast
- Efficient for large data
- Low computational overhead
- Suitable for real-time applications

Disadvantages

- Secure key distribution is difficult.
- Every communicating party must protect the same secret key.

Common Symmetric Algorithms

AES (Advanced Encryption Standard)

The global standard for symmetric encryption.

Key sizes:

- AES-128
- AES-192
- AES-256

Used in:

- HTTPS
- VPNs
- Disk encryption
- Cloud storage
- Government systems

AES is currently considered secure against classical attacks. Quantum computers may reduce its effective security through Grover's algorithm, but AES-256 remains a strong long-term choice.

DES

Developed in the 1970s.

Key size:

56 bits

Now considered insecure due to brute-force attacks.

Triple DES (3DES)

An extension of DES that applies the algorithm three times.

More secure than DES but now deprecated due to performance limitations and security concerns.

ChaCha20

A modern stream cipher designed for high performance, especially on devices without hardware AES acceleration.

Often paired with Poly1305 for authenticated encryption.

---

8. Asymmetric Cryptography

Asymmetric cryptography uses **two mathematically related keys**.

- Public Key
- Private Key

```
Public Key
     |
Encrypt
     |
Ciphertext
     |
Decrypt
     ▼
Private Key
```

The public key can be shared openly.

The private key must remain secret.

Advantages

- Solves the key distribution problem.
- Enables digital signatures.
- Enables secure communication between strangers.

Disadvantages

- Slower than symmetric encryption.
- Computationally intensive.

RSA

Based on integer factorization.

Widely used for:

- TLS
- Certificates
- Digital signatures
- Email encryption

Quantum computers running Shor's algorithm could efficiently factor large integers, making RSA insecure in a post-quantum world.

Elliptic Curve Cryptography (ECC)

Provides equivalent security to RSA with much smaller key sizes.

Common curves:

- P-256
- P-384
- Curve25519

Used in:

- Mobile devices
- Blockchain
- TLS
- SSH

ECC is also vulnerable to Shor's algorithm.

---

9. Hybrid Cryptography

Modern communication rarely relies on a single cryptographic technique.

Instead, systems combine asymmetric and symmetric cryptography.

Example: HTTPS

1. Browser authenticates the server using RSA or ECC certificates.
2. A temporary symmetric session key is established.
3. All subsequent communication uses AES or ChaCha20.

This provides the performance of symmetric encryption with the convenience of asymmetric key exchange.

---

10. Hash Functions

Hash functions transform input data into a fixed-length output called a digest.

Properties:

- Deterministic
- One-way
- Collision-resistant
- Fast to compute

Example:

```
Input:
hello

SHA-256

↓

2cf24dba5fb0...
```

Uses

- Password storage
- File integrity
- Digital signatures
- Blockchain
- Software verification

Common Hash Algorithms

MD5

Produces a 128-bit digest.

Broken due to collision attacks.

Should not be used for security-critical applications.

SHA-1

Produces a 160-bit digest.

Deprecated because practical collision attacks exist.

SHA-256

Part of the SHA-2 family.

Currently one of the most widely trusted cryptographic hash functions.

SHA-3

Designed using the Keccak construction.

Provides an alternative to SHA-2 with different internal design principles.

---

11. Message Authentication Codes (MAC)

A MAC ensures both integrity and authenticity by combining a secret key with a hash function.

Example:

HMAC-SHA256

Used in:

- APIs
- Cloud authentication
- Payment gateways

Unlike plain hashes, only parties with the secret key can generate or verify a valid MAC.

---

12. Digital Signatures

Digital signatures provide:

- Authentication

- Integrity
- Non-repudiation

Process:

1. Hash the message.
2. Sign the hash with the sender's private key.
3. Verify using the sender's public key.

Widely used for:

- Software signing
- PDF signing
- Secure email
- Government documents
- Enterprise identity

---

13. Random Number Generation

Strong cryptography depends on unpredictable randomness.

Weak randomness leads to predictable keys.

Types:

- True Random Number Generator (TRNG)
- Pseudorandom Number Generator (PRNG)
- Cryptographically Secure PRNG (CSPRNG)

Enterprise systems should always use CSPRNGs for generating cryptographic keys and nonces.

---

14. Key Management

Keys are the most valuable assets in any cryptographic system.

A secure algorithm with a compromised key provides no protection.

Key lifecycle:

```
Generate
    ↓
Store
    ↓
Distribute
    ↓
```

```
    Use
        ↓
    Rotate
        ↓
    Archive
        ↓
    Destroy
```

Enterprise key management involves:

- Secure generation
- Hardware-backed storage (HSM)
- Rotation policies
- Revocation
- Audit logging

---

15. Public Key Infrastructure (PKI)

PKI is the framework that enables trusted digital identities.

Core components:

- Root Certificate Authority (Root CA)
- Intermediate Certificate Authorities
- End-Entity Certificates
- Certificate Revocation Lists (CRLs)
- Online Certificate Status Protocol (OCSP)

PKI underpins:

- HTTPS
- VPNs
- Enterprise authentication
- Secure email
- Code signing

ECDAT must be able to discover and analyze these components across an organization.

---

16. Certificates and Certificate Authorities

A digital certificate binds a public key to an identity.

It typically contains:

- Subject
- Issuer
- Public Key
- Signature Algorithm
- Validity Period
- Extensions

Certificates are signed by trusted Certificate Authorities (CAs).

ECDAT should identify:

- Expired certificates
- Weak signature algorithms
- Short key lengths
- Unsupported cryptographic algorithms

---

17. Secure Communication Protocols

Cryptography is implemented through protocols.

Important examples include:

- HTTPS
- TLS
- SSH
- IPSec
- VPN
- S/MIME
- DNSSEC

Each protocol combines encryption, authentication, and integrity mechanisms.

Understanding where these protocols are used is essential for cryptographic discovery.

---

18. Cryptographic Libraries

Developers rarely implement cryptography directly.

Instead, applications use trusted libraries.

Common libraries include:

- OpenSSL
- BoringSSL
- LibreSSL
- libsodium
- Bouncy Castle
- Java Cryptography Architecture (JCA)
- Microsoft CNG

ECDAT should identify:

- Library versions
- Deprecated APIs
- Vulnerable implementations
- Embedded cryptographic code

---

19. Cryptography in Enterprise Systems

Cryptography appears throughout modern enterprises, often in places developers overlook.

Examples include:

- Source code repositories
- Configuration files
- CI/CD pipelines
- Docker images
- Kubernetes Secrets
- API gateways
- Databases
- Backup systems
- Identity providers
- Cloud Key Management Services
- Hardware Security Modules
- TLS termination proxies
- VPN gateways
- Mobile applications
- Firmware
- Third-party libraries

A comprehensive discovery platform must inventory cryptographic usage across all of these environments.

---

20. Common Cryptographic Weaknesses

Organizations frequently encounter issues such as:

- Hardcoded encryption keys
- Weak random number generation
- Deprecated algorithms (DES, 3DES, MD5, SHA-1)
- Expired certificates
- Improper key rotation
- Insecure protocol versions (SSLv2, SSLv3)
- Misconfigured TLS
- Self-signed certificates in production
- Unencrypted sensitive data
- Poor secrets management

These weaknesses create operational and security risks that ECDAT aims to identify.

---

21. Why Enterprises Need Cryptographic Discovery

Large organizations often do not have a complete inventory of their cryptographic assets.

This leads to questions such as:

- Where is RSA still used?
- Which applications depend on SHA-1?
- Which certificates expire next month?
- Which systems contain hardcoded secrets?
- Which cloud services rely on legacy cryptographic libraries?
- Which applications protect data that must remain confidential for decades?

Without visibility, organizations cannot effectively migrate to post-quantum cryptography.

ECDAT addresses this challenge by creating a comprehensive inventory, assessing quantum risk, and recommending migration strategies.

---

22. Phase Summary

This phase established the foundational concepts required for understanding the ECDAT problem statement.

Key takeaways include:

- Cryptography is essential for securing modern digital systems.
- Symmetric and asymmetric cryptography serve complementary roles.
- Hash functions, MACs, and digital signatures provide integrity, authentication, and non-repudiation.
- Key management and PKI are central to enterprise security.
- Cryptographic functionality is distributed across applications, infrastructure, cloud platforms, and development pipelines.
- Legacy algorithms, poor key management, and hidden cryptographic dependencies create significant security and operational risks.
- Enterprise cryptographic discovery is the prerequisite for assessing quantum readiness and planning migration to post-quantum cryptography.

With these fundamentals in place, the next phase will explore **Quantum Computing**, why it threatens current public-key cryptography, and how those emerging capabilities drive the need for solutions like ECDAT.